

COURSE CODE: DCIT 208

ASSIGNMENT 1

STUDENT NAME: SAMUEL PETER

STUDENT ID: 22410937

TEAM REPOSITORY URL: https://github.com/Dept-of-Comp-Sci-University-of-Ghana/dcit208-client-project-2026-team-engineering-repository-seg26-74-nextgen_dev

INDIVIDUAL PORTFOLIO REPOSITORY URL: [Samuelpeterr/DCIT208-Portfolio: My individual portfolio repository](#)

YOUTUBE LINK: <https://youtu.be/ytf0cDczgDo>

TEAM NAME: NEXT GEN DEVELOPERS

PROJECT TITLE: ABESDAC CONNECT

REQUEST FLOW ANALYSIS FOR ABESDAC CONNECT – ONLINE OFFERTORY AND DONATION SYSTEM

Chosen Feature: Online Offertory and Donation System

ABESDAC CONNECT is a church website created to improve communication, participation, and digital service delivery between the church and its members. One major feature of the platform is an Online Offertory and Donation System that allows members and supporters to make financial contributions electronically through the website.

This document explains the complete request flow for the donation process and describes how requests move through the system from the user's device to the final response.

1. What Device Is the User On?

The user accesses ABESDAC CONNECT using either a desktop computer, laptop, tablet, or smartphone (Iphone or android) connected to the internet.

Because this feature is implemented as a website, the primary access method is through a web browser. Supported browsers may include Chrome, Safari, Firefox, Edge, and similar modern browsers.

The browser acts as the client device and is responsible for displaying pages, collecting user input, sending requests, and presenting server responses.

2. Is It a Browser or Mobile App?

For this project, the donation system operates through a browser-based web application rather than a dedicated mobile application.

Members visit the website directly and interact with the donation interface through browser pages.

This approach allows users to access the system without installing software.

3. What URL, Route, Screen, or Endpoint Is Involved?

The donation process begins when users open the website.

Example website:

<https://www.abesdacconnect.org>

The user navigates to:

/donate

Example full route:

<https://www.abesdacconnect.org/donate>

After entering donation information and clicking submit, the browser sends a request to an application endpoint.

Example endpoint:

POST /api/donations

This endpoint receives donation information and begins server-side processing.

The donation page may contain:

- Donor name field
- Donation amount field
- Donation category
- Payment details
- Submit button

4. What Protocol Is Used?

The system uses HTTPS.

HTTPS stands for HyperText Transfer Protocol Secure.

HTTPS protects communication between the user and server through encryption.

This is especially important because the donation process may involve personal and financial information.

The browser first performs DNS resolution to identify the correct server address and then establishes a secure encrypted connection before transmitting data.

5. What Data Is Sent?

When the user submits the donation form, the browser packages information into an HTTP request body.

Example data sent:

```
{  
  donorName,  
  email,  
  phoneNumber,  
  donationAmount,  
  donationType,  
  paymentMethod,  
  transactionTimestamp  
}
```

Example values:

Name: Samuel Peter

Amount: GHS 100

Donation Type: Offertory

Payment Method: Card or Mobile Money

Sensitive information should not be stored directly unless required.

6. What Validates the Request?

Request validation occurs in two stages.

Client-side validation:

The browser checks whether required fields are completed before submission.

Examples:

- Empty amount field
- Invalid email format
- Negative donation amount

Server-side validation:

The application server performs deeper checks after receiving the request.

Examples:

- Confirm amount is numeric
- Prevent duplicate transactions
- Verify payment format
- Confirm request structure

Server-side validation is important because client-side checks alone are not secure.

7. What Database Table, Collection, or External Service Is Touched?

Once validation succeeds, the application communicates with both internal storage and external services.

Database Table:

donations

Example columns:

donation_id

member_id

donation_amount

payment_method

status

created_at

Additional table:

transactions

Example fields:

transaction_id

gateway_reference

approval_status

External Service:

Payment Gateway

Examples may include:

- Mobile Money service
- Bank payment processor
- Online payment provider

The external service processes payment authorization and returns transaction results.

8. What Response Comes Back?

After processing is completed, the server returns a response to the browser.

Successful response:

Status Code: 200

Message:

“Donation Successful”

Additional details:

Transaction ID

Amount Received

Confirmation Message

The browser then displays a confirmation page and optionally generates a digital receipt.

9. What Happens if Validation Fails?

If validation fails, processing immediately stops.

Example failures:

- Missing donor information
- Invalid amount
- Incorrect payment details

Example response:

Status Code: 400

Message:

“Invalid donation request. Please correct the highlighted fields.”

The user remains on the donation page and receives instructions to fix errors.

No payment attempt is made.

10. What Happens if the Database Is Down?

Database failure is a critical failure point.

If the application cannot save transaction information:

- Donation confirmation cannot be completed
- Transaction records may become unavailable
- Users may receive an error notification

Possible response:

Status Code: 503

Message:

“Service temporarily unavailable.”

Possible mitigation strategies include:

- Retry mechanisms
- Temporary request queues
- Backup databases
- Transaction recovery logs

11. What Happens if 500 Users Do This at Once?

Large traffic volume introduces scalability challenges.

If 500 users attempt donations simultaneously:

Potential problems:

- Increased server response time
- Database congestion
- Payment delays
- System timeouts

Possible solutions:

Load balancing:

Distribute requests across multiple servers.

Caching:

Reduce repeated processing.

Database optimization:

Improve query performance.

Cloud scaling:

Increase server resources automatically.

Queue systems:

Process transactions in controlled order.

These methods improve reliability during fundraising events or large church services.

12. What Security or Privacy Concern Exists?

Security is one of the most important requirements because financial information is involved.

Major concerns include:

Unauthorized access:

Attackers attempting to view donor information.

Data interception:

Sensitive data captured during transmission.

Duplicate transactions:

Users accidentally submitting multiple donations.

Privacy exposure:

Unauthorized viewing of donor records.

Protection measures:

- HTTPS encryption
- Secure authentication
- Role-based access control
- Input validation
- Transaction logging
- Database permissions
- Session protection

Sensitive payment information should not be exposed publicly.

Conclusion

The Online Offertory and Donation System in ABESDAC CONNECT demonstrates how modern web applications process user requests securely and efficiently. Beginning from the user's browser and continuing through validation, payment processing, database storage, and response generation, each stage contributes to a reliable donation experience. By implementing scalability, error handling, and security measures, the platform aims to support church operations and provide members with a safe and convenient digital giving environment.

I used / did not use AI assistance. If used, the tool, purpose, prompt summary, output accepted/rejected and verification steps are documented. I understand that I must be able to explain and defend every part of this submission.